Intern Assignment Test

Full-Stack AI Developer

Project: AI-Powered Accounting & Finance Assistant

Role	Full-Stack AI Developer (Intern)
Assignment Type	Research + Development + Deployment
Methodology	Spec-Driven Development (SDD)
Submission Deadline	Wednesday, 29 July 2026 (hard deadline — no extensions)
Submission Rule	Submit whatever you have completed by the deadline. Partial work is accepted, but submission on the deadline is mandatory.

1. Assignment Overview

You are required to design, research, build, and deploy a **full-stack web application with an integrated AI service** that automates the day-to-day work of an accountant / chartered accountant (CA). The application will allow a user (e.g., an office admin or business owner) to manage financial records — daily expenses, monthly office expenses, income entries, and so on — through a modern web UI, while an **AI agent** automates accounting tasks on the user's behalf.

The AI service is the heart of this project. The user should be able to interact with the AI in natural language to perform accounting operations such as: adding or updating entries, generating a Profit & Loss statement, preparing a balance sheet, running an audit for any given month, summarizing spending patterns, answering questions like "How much did we spend on utilities in March?", and any other task an accountant typically performs.

Important: We are intentionally NOT giving you a fixed list of accounting features. Discovering *which accounting/CA tasks exist* and *how each one can be automated with AI* is part of your research. Your research quality will be evaluated as seriously as your code.

2. Assignment Phases

Phase 1 — Research & Research Paper

Before writing any code, conduct proper research and produce a **research paper** (PDF). Your research paper must cover at minimum:

- What are the core responsibilities and daily/monthly/yearly tasks of an accountant or CA? (e.g., bookkeeping, ledgers, journal entries, expense tracking, balance sheet, profit & loss statement, trial balance, cash flow, audits, tax summaries, reconciliation, etc.)
- Which of these tasks can realistically be automated using AI, and how? Map each task to an automation approach (e.g., natural-language entry creation, report generation, anomaly detection in audits, categorization of expenses).
- How AI agents / agentic frameworks work, and which agentic framework you selected and why (comparison of at least 2–3 options).

- Which AI model(s) you will use and why (cost, capability, free-tier availability, etc.).
- Your proposed system architecture: frontend, backend, database, AI layer, and how they communicate.
- The exact list of features you will implement in this project, derived from your research.
- References / sources you used.

Recommended length: 6–15 pages. It must be your own writing and understanding. If you use AI tools to assist your research, you must still be able to explain and defend every part of it in the review meeting.

Phase 2 — Specification & Workflow Design (SDD)

This project must follow **Spec-Driven Development (SDD)**: write clear specifications first, then implement against those specs. Your specs should define features, API contracts, data models, and AI-agent behavior before implementation.

- Create a complete **workflow diagram** of the system on **Lucidchart** or **draw.io** (this tool choice is a requirement, not optional) — covering user flows, AI agent flow (how a user request travels from UI → API → agent → tools → database → response), and data flow. The diagram's **shareable URL** must be submitted with your deliverables.
- Write feature specs (what each feature does, inputs, outputs, edge cases) before coding it.
- Keep your specs in the repository (e.g., a /specs folder) so we can see that development followed the specs.

Phase 3 — Development

Build the application according to the required tech stack (Section 3). Expected capabilities:

- **UI (Next.js + TypeScript):** screens for daily expenses, monthly office expenses, income, ledgers/records list, reports (P&L, balance sheet, audit view), and an AI chat/assistant interface.
- **Manual + AI-driven data entry:** the user can add entries through forms, and also by simply telling the AI (e.g., "Add office rent 50,000 for July").
- **AI automation:** the agent must be able to perform accounting operations — create/read/update entries, generate a Profit & Loss statement, prepare a balance sheet, run a monthly audit, answer any accounts-related question from the data, and other tasks identified in your research.
- **Reports:** generate financial statements from real data stored in PostgreSQL (not hard-coded).
- **Validation:** all API request/response models must use Pydantic.

Phase 4 — Deployment

- Deploy the full project (frontend + backend + database) using any **free** hosting services (e.g., Vercel for Next.js; Railway / Render / Hugging Face Spaces / Fly.io for FastAPI; Neon / Supabase / Railway for PostgreSQL — your choice).
- The live deployed link must be working at submission time.
- A complete **Docker setup** (Dockerfile(s) and docker-compose) must be included so the whole project can be run locally with Docker.

3. Required Tech Stack

Layer	Requirement
Frontend	Next.js with TypeScript
Backend	Python, managed with uv (package/environment manager)
API Framework	FastAPI
Data Validation	Pydantic models (for all request/response schemas)
Database	PostgreSQL
AI Model	Developer's choice — any model (free-tier friendly options are fine)
Agentic Framework	Any code-based agentic framework (e.g., OpenAI Agents SDK, LangGraph, CrewAI, etc.) — your choice, justified in your research paper
Methodology	Spec-Driven Development (SDD)
Workflow Diagram Tool	Lucidchart or draw.io (mandatory) — the diagram's shareable URL must be submitted along with the other deliverables
Containerization	Docker (Dockerfile + docker-compose)

4. GitHub & Version Control Requirements

- Work in a public (or shared-access) GitHub repository and submit the repo URL.
- **Every feature must be developed on its own branch** (e.g., feature/expense-entry, feature/ai-agent, feature/pl-report) and merged into the main branch via pull requests.
- Commits must be **frequent, small, and meaningful** with proper commit messages (e.g., "feat: add monthly audit endpoint", "fix: expense date validation"). A single giant "final commit" will be heavily penalized.
- Repository must include: a clear README (setup + run instructions), the /specs folder (SDD documents), the workflow diagram (image/PDF or link), and the Docker setup.

5. What You Must Submit

#	Deliverable	Details
1	**Live Deployment Link**	Working URL of the deployed application (frontend connected to backend and database).
2	**Research Paper (PDF)**	As described in Phase 1.
3	**Workflow Diagram URL**	Made on Lucidchart or draw.io (mandatory). Share the diagram's public/shareable URL, and also keep an export (PNG/PDF) in the repo.
4	**GitHub Repository URL**	With feature branches, proper commits, README, and specs.
5	**Docker Setup**	Dockerfile(s) + docker-compose so the project runs with one command.
6	**AI Chat History**	Chat history of every AI tool used (Claude, Gemini CLI, Codex, etc.) — all prompts given for any task or coding work.

6. Deadline & Submission Policy

- **Deadline: Wednesday, 29 July 2026.** Submission on this date is mandatory for everyone.
- There are **no extensions**. Submit as much as you were able to complete — a partially finished project submitted on time is acceptable; a late submission is not.
- Along with your links, include a short note (in the README or email) stating what is done, what is partially done, and what is pending.

7. Guidance & Suggested Order of Work

- **Days 1–2:** Research accounting/CA tasks and AI automation approaches; write the research paper; finalize your feature list.
- **Day 2:** Draw the full workflow on Lucidchart/draw.io; write your specs (SDD); design the database schema.
- **Days 3–4:** Set up the repo, Docker, PostgreSQL, FastAPI backend (with Pydantic models), and core CRUD APIs. Start the Next.js UI in parallel.
- **Day 4–5 (early):** Integrate the AI agent with tools that call your APIs/database; implement reports (P&L, balance sheet, monthly audit).
- **Day 5 (Friday):** Deploy everything, test the live link, clean up the README, and submit.

Tip: Get a thin end-to-end slice working early (one expense entry created via AI, stored in PostgreSQL, shown in the UI), then expand feature by feature — each on its own branch.

Questions about the assignment are welcome before the deadline. Plagiarized submissions (copied repositories or papers) will be rejected. You must be able to explain every part of your research and code in the review. Good luck — we are excited to see what you build.